# k-Nearest Neighbor Learning(*k*NN)

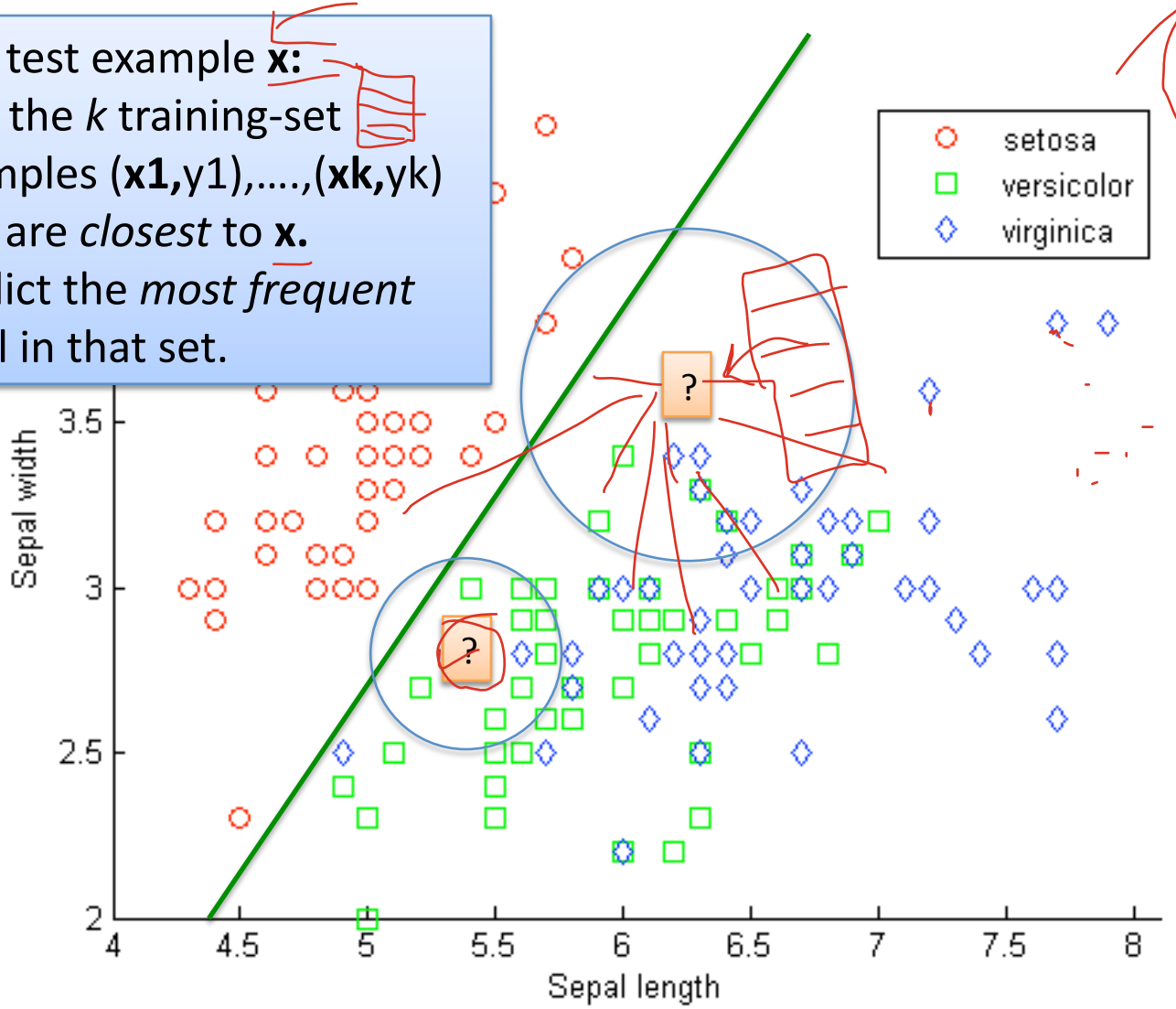
Dr. Panthadeep Bhattacharjee
Dept. of CSE
NIT Rourkela

# Nearest Neighbor Learning

# k-nearest neighbor learning

Given a test example **x:**

1. Find the *k* training-set examples (**x1,**y1),….,(**xk,**yk) that are *closest* to **x.**
2. Predict the *most frequent* label in that set.



# Breaking it down:

- To train:
  - save the data
- To test:
  - For each test example **x:**

Very fast!

…you might build some indices….

1. Find the *k* training-set examples (**x1**,y1), ….,(**xk**,yk) that are *closest* to **x.**
2. Predict the *most frequent* label in that set.

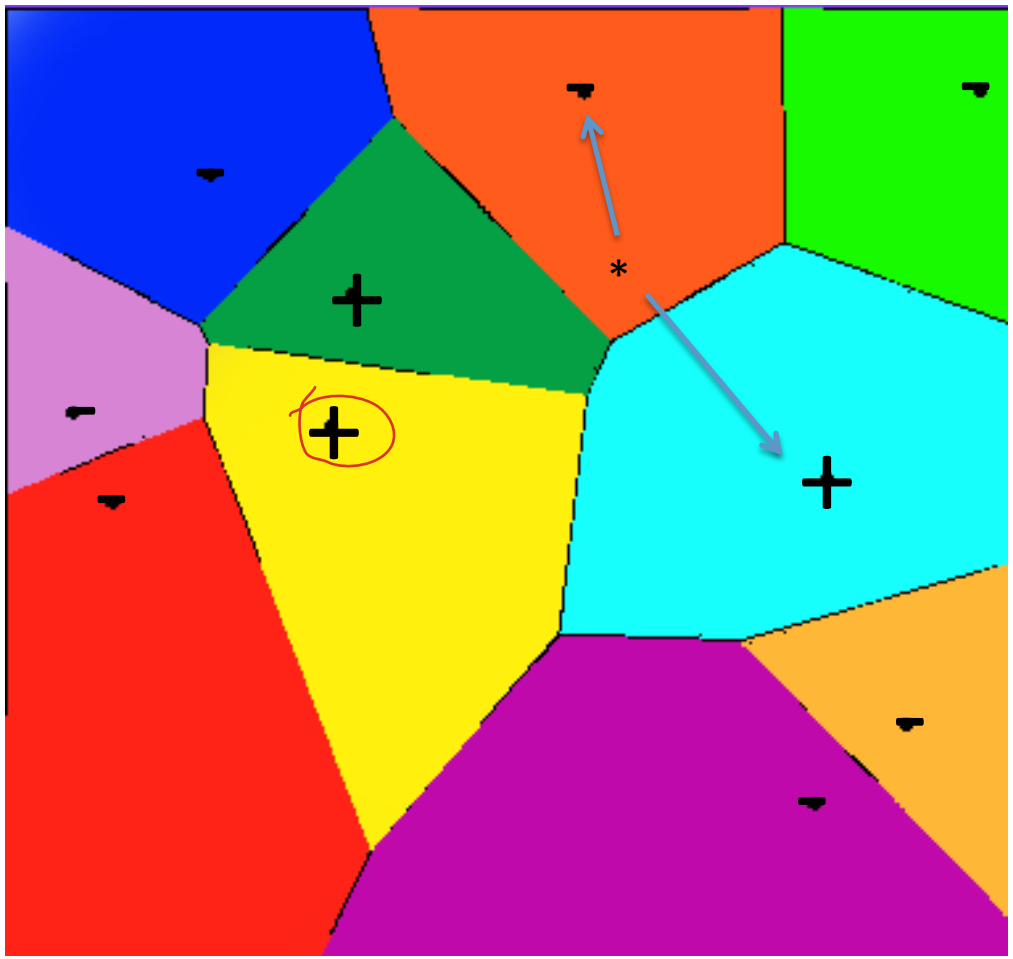
Prediction is relatively slow (compared to a linear classifier or decision tree)

?

# What is the decision boundary for 1-NN?

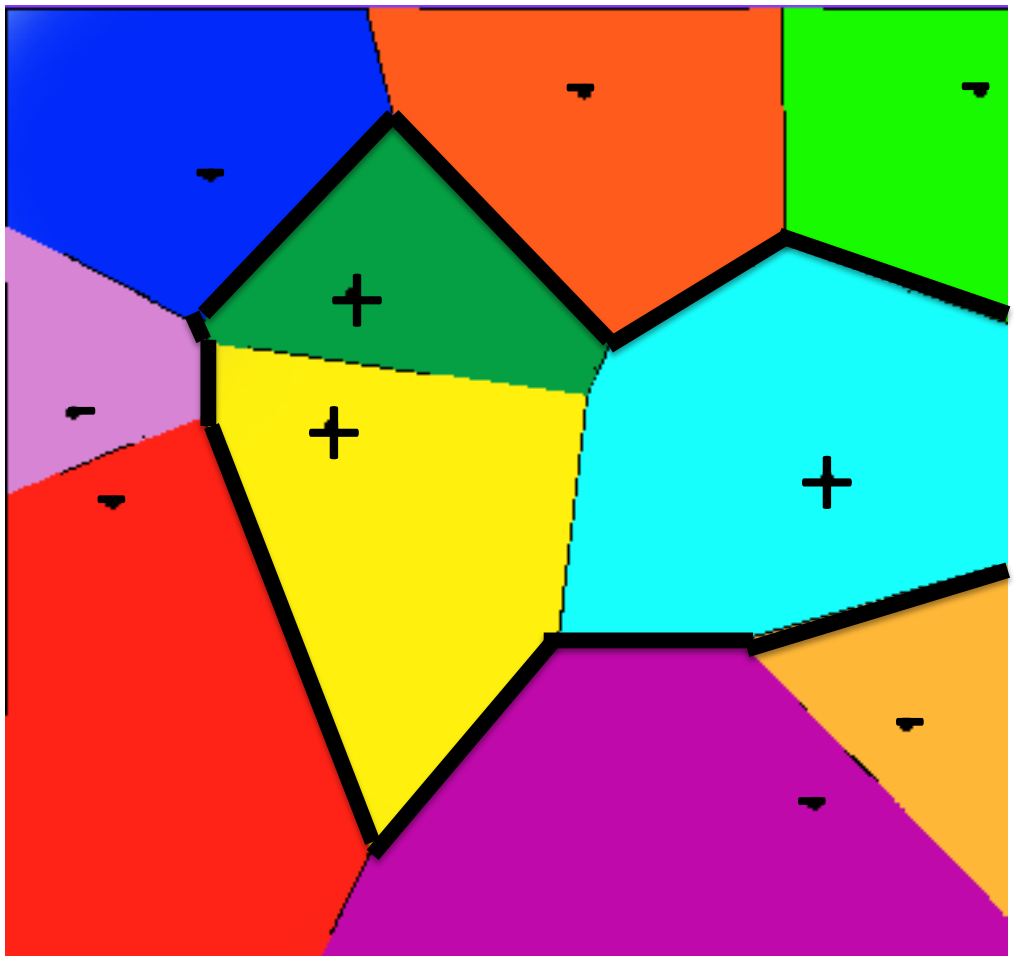
Voronoi Diagram

Each cell $C_i$ is the set of all points that are closest to a particular example $\mathbf{x}_i$

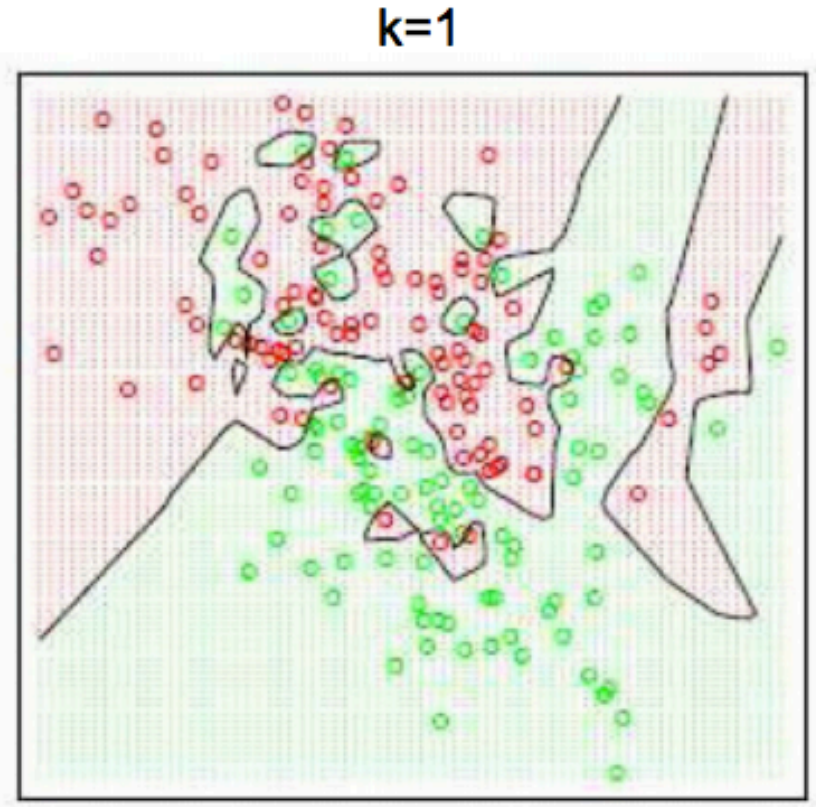


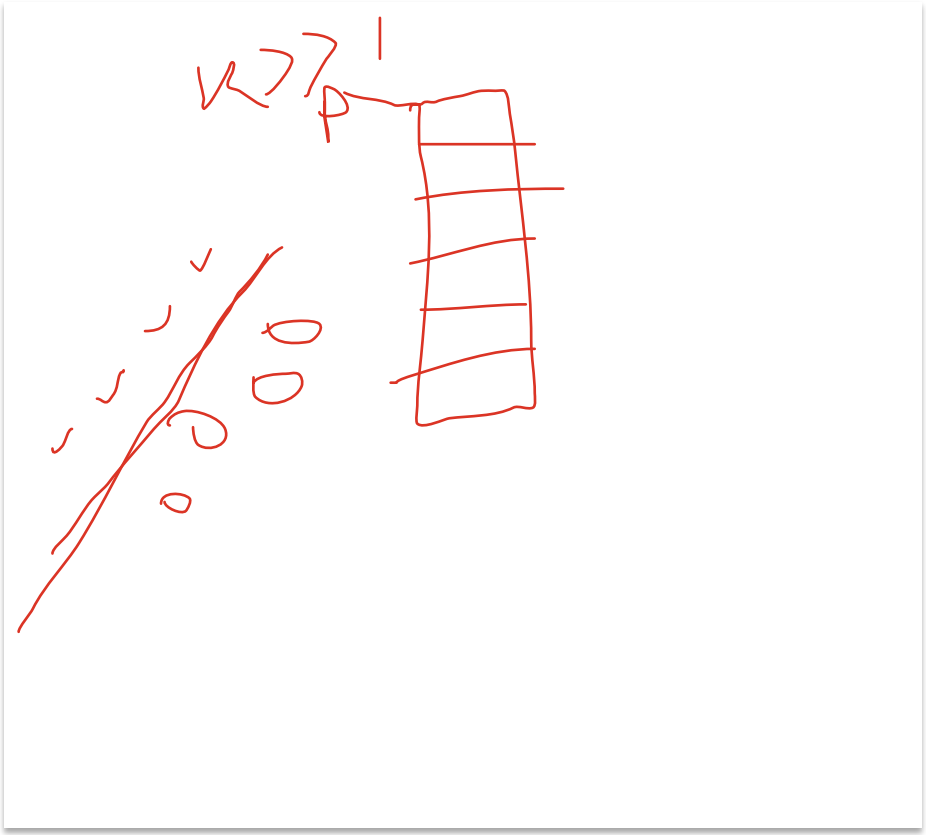
# What is the decision boundary for 1-NN?

Voronoi Diagram



# Effect of k on decision boundary



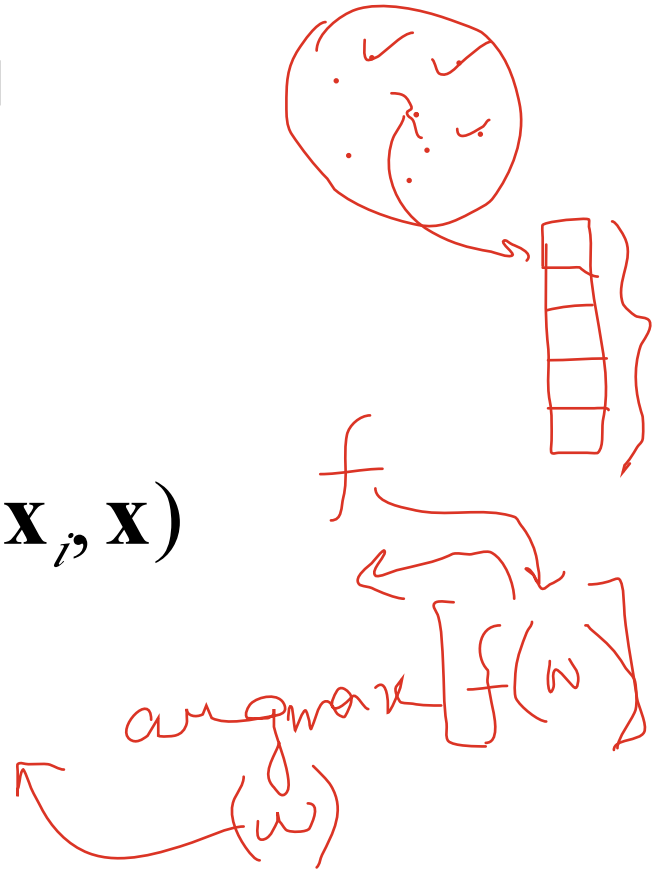



Figures from Hastie, Tibshirani and Friedman (Elements of Statistical Learning)

# Some common variants

- Distance metrics:
  - Euclidean distance: **||x1 - x2||**
  - Cosine distance: 1 - **<x1,x2>/||x1||*||x2||**
    - this is in [0,1]
- Weighted nearest neighbor:
  - Instead of most frequent *y* in k-NN predict

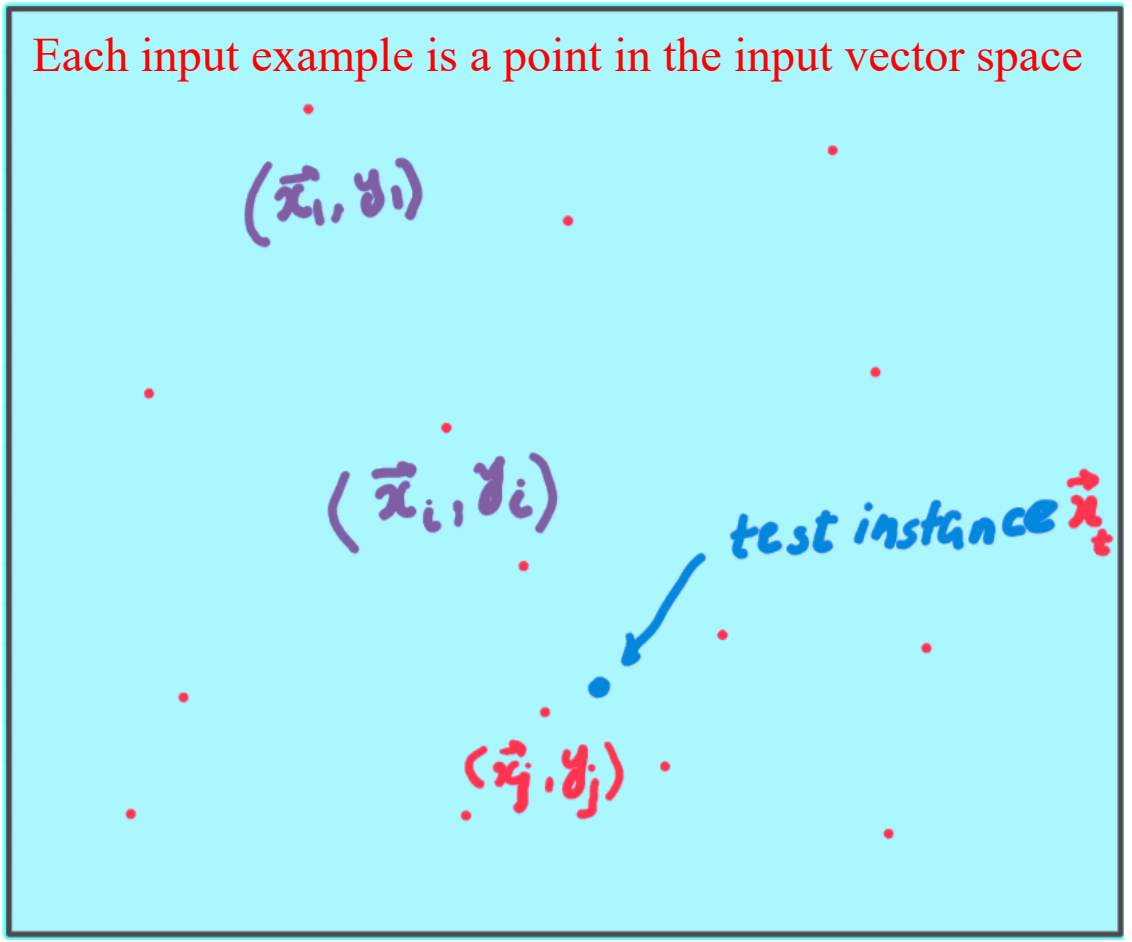
$$\text{argmax}_y \sum_{(\mathbf{x}_i, y) \in kNN(\mathbf{x})} sim(\mathbf{x}_i, \mathbf{x})$$



# Instanced based learning

- The **model is not learned a-priori** from the training set to predict the test-example's output.
  - Store the training examples (don't learn)
  - If test instance is given, then try to predict its output from the stored data
- Lazy learning

# 1-Nearest Neighbor




**Training Phase:**

Save the examples

**Testing Phase**

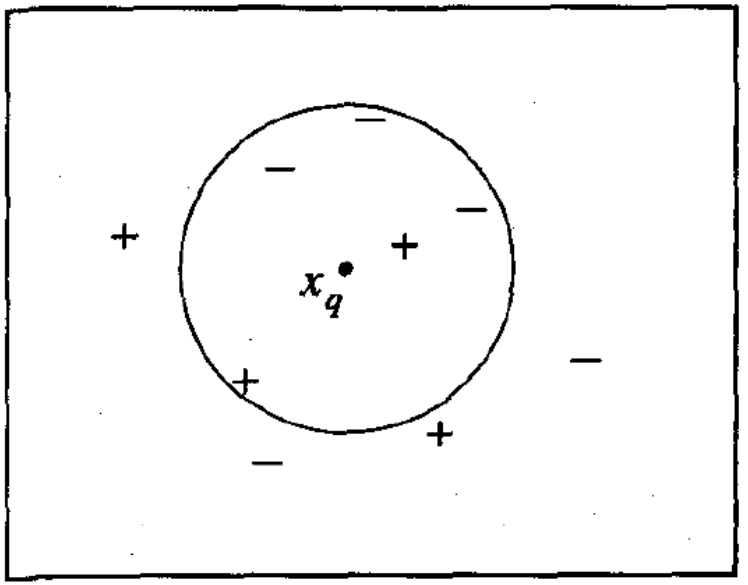
Get the test instance $x_q$

Find the most similar training example $(x_j, y_j)$

(the one closest to $x_q$)

Predict $y_j$ as the output $y_q$

# Basic k-Nearest Neighbor




**Training Phase:**

Save the examples in some suitable data structure

**Testing Phase**

Get the test instance $x_q$

Find the **k** most similar training examples $\{(x_1, y_1), \ldots, (x_k, y_k)\}$ (those are closest to $x_q$)

Predict as the output $y_q$ ???

**Classification:** Predict the majority class from $\{y_1, y_2, \ldots, y_k\}$

**Regression:** Predict the average of $\{y_1, y_2, \ldots, y_k\}$

# kNN – mathematical description

- $(x_i, f(x_i))$ – list of training examples
- Attributes of Instance x:

$$\langle a_1(x), a_2(x), \ldots a_n(x) \rangle$$

- The Euclidean distance between two instances $x_i$ and $x_j$

$$d(x_i, x_j) \equiv \sqrt{\sum_{r=1}^{n} (a_r(x_i) - a_r(x_j))^2}$$

## k-NN for approximating a discrete-valued function $f: R^n \rightarrow V$, where $V = \{v_1, v_2, \ldots, v_s\}$

---

Training algorithm:

- For each training example $\langle x, f(x) \rangle$, add the example to the list *training_examples*

Classification algorithm:

- Given a query instance $x_q$ to be classified,
  - Let $x_1 \ldots x_k$ denote the $k$ instances from *training_examples* that are nearest to $x_q$
  - Return

$$\hat{f}(x_q) \leftarrow \underset{v \in V}{\operatorname{argmax}} \sum_{i=1}^{k} \delta(v, f(x_i))$$

where $\delta(a, b) = 1$ if $a = b$ and where $\delta(a, b) = 0$ otherwise.

---

- **k-NN for approximating a real valued function $f: R^n \rightarrow R$**

---

Training algorithm:

- For each training example $\langle x, f(x) \rangle$, add the example to the list *training_examples*

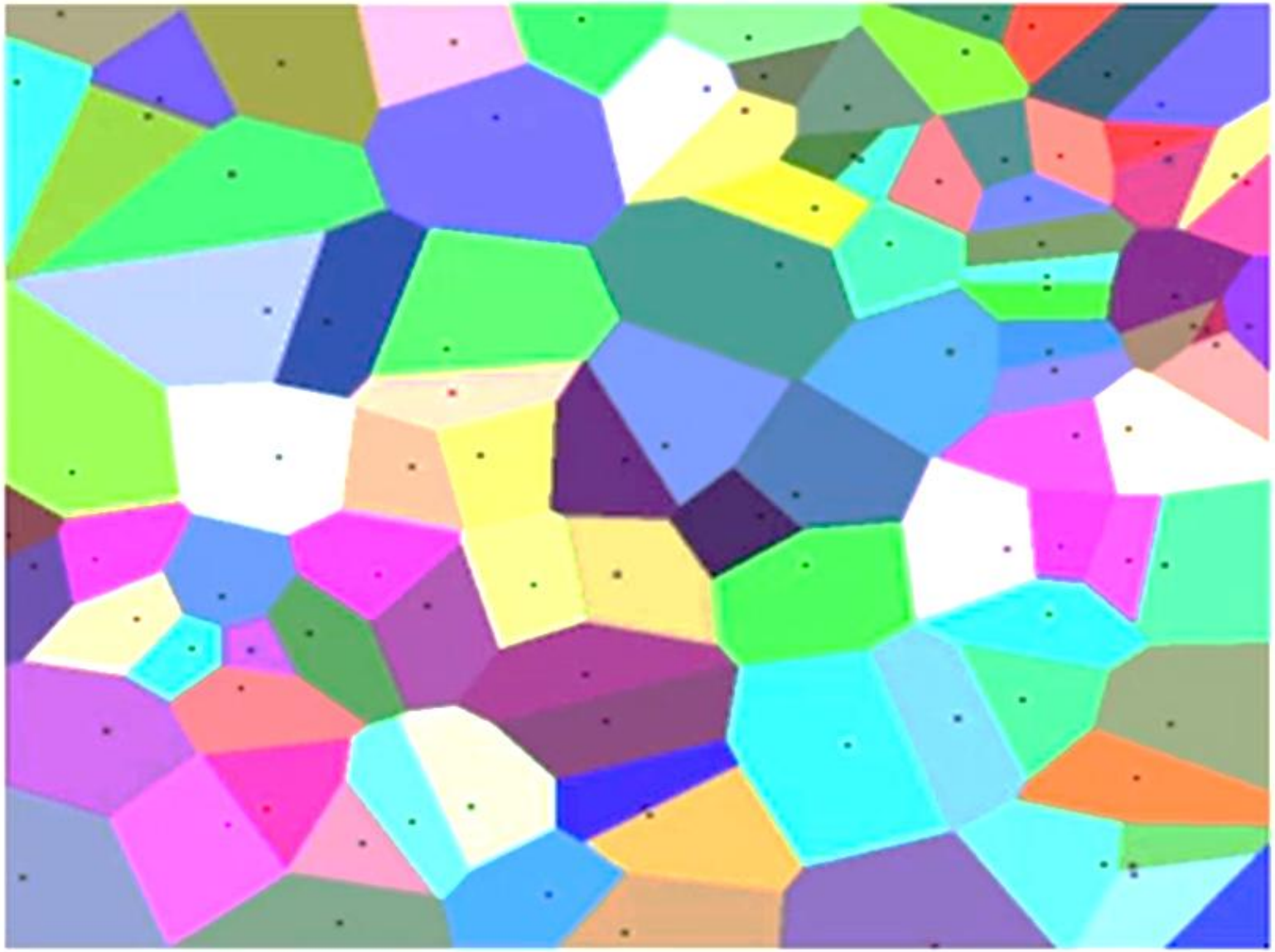
Classification algorithm:

- Given a query instance $x_q$ to be classified,
  - Let $x_1 \ldots x_k$ denote the $k$ instances from *training_examples* that are nearest to $x_q$
  - Return

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} f(x_i)}{k}$$

# Decision Boundary

- 1-NN Voronoi diagram



# k-NN learning for classification – Example

| Sl. | Height | Weight | Target | Euclidean dist |
|---|---|---|---|---|
| 1 | 150 | 50 | Medium | 8.06 |
| 2 | 155 | 55 | Medium | 2.24 |
| 3 | 160 | 60 | Large | 6.71 |
| 4 | 161 | 59 | Large | 6.40 |
| 5 | 158 | 65 | Large | 11.05 |
| 6 | 157 | 54 | ?? | |

$\hat{f}(x)\ for\ v = Medium\ is\ 1$
$\hat{f}(x)\ for\ v = Large\ is\ 2$
So output is “Large”

# k-NN learning for regression – Example

| Sl. | Height | Weight | Target | Euclidean dist |
|---|---|---|---|---|
| 1 | 150 | 50 | 1.5 | 8.06 |
| 2 | 155 | 55 | 1.2 | 2.24 |
| 3 | 160 | 60 | 1.8 | 6.71 |
| 4 | 161 | 59 | 2.1 | 6.40 |
| 5 | 158 | 65 | 1.7 | 11.05 |
| 6 | 157 | 54 | ?? | |

$$\hat{f}(x_q) = {}^{(1.2+1.8+2.1)}/_{3} = 1.7$$

So output is 1.7

# More on Euclidean Distance

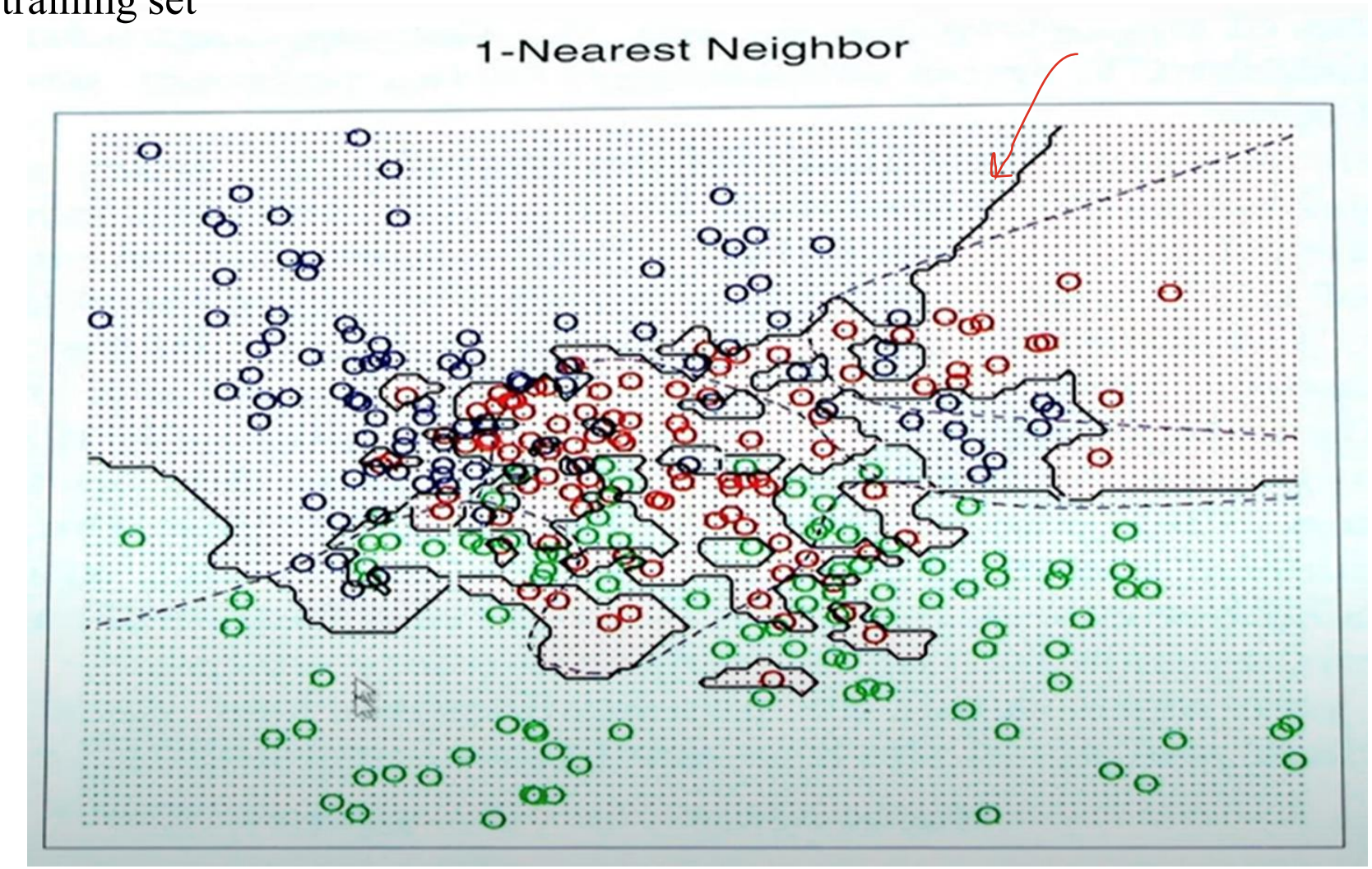
$$d(x_i, x_j) \equiv \sqrt{\sum_{r=1}^{n} (a_r(x_i) - a_r(x_j))^2}$$

- When can we assign equal weights to the attributes
  - Only if the scale of the attributes and the differences are similar
  - OR attributes are scaled to almost equal range

- What if the attributes are not scaled
- What if one attribute has more noise than other
  - Use larger ***k*** to smooth out the differences and
  - Use weighted distance functions

# Small vs. large *k*

- Small k captures the fine structure of problem space
  - May be necessary for small training set
- Large k
  - Less sensitive to class noise
  - Larger training set allows to use large k



# Weighted Euclidean distance

$$D(x_p\ x_q) = \sqrt{\sum_{i=1}^{n} w_i . \left(a_i(x_p) - a_i(x_q)\right)^2}$$

- Large weights =>  attribute is more important
- Small weights => attribute is less important
- Zero weight => attribute doesn’t matter

# Distance weighted k-NN for discrete valued $f$

Training algorithm:

- For each training example $\langle x, f(x) \rangle$, add the example to the list *training_examples*

Classification algorithm:

- Given a query instance $x_q$ to be classified,
  - Let $x_1 \ldots x_k$ denote the $k$ instances from *training_examples* that are nearest to $x_q$
  - Return

$$\hat{f}(x_q) \leftarrow \underset{v \in V}{\operatorname{argmax}} \sum_{i=1}^{k} w_i \delta(v, f(x_i))$$

where

$$w_i \equiv \frac{1}{d(x_q, x_i)^2}$$

# Distance weighted k-NN for real valued $f$

---

Training algorithm:

- For each training example $\langle x, f(x) \rangle$, add the example to the list *training_examples*

Classification algorithm:

- Given a query instance $x_q$ to be classified,
  - Let $x_1 \ldots x_k$ denote the $k$ instances from *training_examples* that are nearest to $x_q$
  - Return

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} w_i f(x_i)}{\sum_{i=1}^{k} w_i}$$

where

$$w_i \equiv \frac{1}{d(x_q, x_i)^2}$$

# Bayesian Learning

Dr. Panthadeep Bhattacharjee
Dept. of CSE
NIT Rourkela

# Bayesian learning

- Bayesian learning is based on probability model
- Given a data "D" objective is to find which hypothesis $h \in H$ provides maximum posteriori probability for the given D
  - Maximum $P(h|D)$
- How to find $P(h|D)$?
  - Bayes Theorem

$A, B$

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \rightarrow (1)$$

$$P(B|A) = \frac{P(A \cap B)}{P(A)} \rightarrow (2)$$

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

$$\Rightarrow P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$$P(B) = \frac{\prod P(B_i|A) \cdot P(A)}{\sum P(B|A_i) P(A_i)}$$

# Bayes Theorem

$$P(h|D) = \frac{P(D|h)\,P(h)}{P(D)}$$

- $P(h)$: the initial probability that hypothesis $h$ holds, before we have observed the training data.
  - $P(h)$ is often called the priorprobability of $h$
- $P(D|h)$ : the probability of observing data D given that hypothesis h holds
- $P(D)$ : the prior probability that training data D will be observed

# maximum a posteriori (MAP) hypothesis

- Chose that $h$ for which $P(h|D)$ is the maximum

$$h_{MAP} \equiv \underset{h \in H}{\operatorname{argmax}} P(h|D)$$

$$= \underset{h \in H}{\operatorname{argmax}} \frac{P(D|h)P(h)}{P(D)}$$

$$= \underset{h \in H}{\operatorname{argmax}} P(D|h)P(h)$$

# Maximum likelihood hypothesis

- In some cases, we assume that before learning any data, the probabilities of all $h \in H$ are same.
- $h_{MAP}$ can now be simplified as follows

$$h_{ML} \equiv \underset{h \in H}{\operatorname{argmax}} P(D|h)$$

- ***P(D|h)*** is often called the ***likelihood*** of the data ***D*** given ***h***

# Bayes Theorem - Example

## Cancer diagnosis problem

- Hypotheses
  - the patient has a particular form of cancer
  - the patient does not have cancer
- Two lab test outcomes
  - Positive ($\oplus$)
  - Negative ($\ominus$)
- Prior knowledge
  - over the entire population of people only 0.8% have some form of cancer
  - In case of actual cancer, the lab test returns a correct positive result in only 98% cases
  - In case of NO cancer, The lab test returns a correct negative result in only 97% cases
- New patient – lab test report $\oplus$
  - Should we diagnose the patient as having cancer or not?

$$P(cancer) = .008, \quad P(\neg cancer) = .992$$
$$P(\oplus|cancer) = .98, \quad P(\ominus|cancer) = .02$$
$$P(\oplus|\neg cancer) = .03, \quad P(\ominus|\neg cancer) = .97$$

- For each $h \in H$ find $P(h|\oplus)$

$$P(\oplus|cancer)\,P(cancer) = (.98).008 = .0078$$
$$P(\oplus|\neg cancer)\,P(\neg cancer) = (.03).992 = .0298$$

- So $h_{MAP} = \neg$ cancer

# Brute-Force MAP Learning Algorithm

1. For each hypothesis $h$ in $H$, calculate the posterior probability

$$P(h|D) = \frac{P(D|h)\,P(h)}{P(D)}$$

2. Output the hypothesis $h_{MAP}$ with the highest posterior probability

$$h_{MAP} = \underset{h \in H}{\operatorname{argmax}}\, P(h|D)$$

- Estimating $\boldsymbol{P(D|h)}$ or $\boldsymbol{P(\vec{X}|C_k)}$ needs huge training set
- Requires significant computation
- Impractical for large hypothesis space

# Naïve Bayes Learning

- Each input instance $\vec{X}$ is described tuple of attribute values $< a_1 , a_2 \ldots a_n >$.
- The target function $f( x )$ can take on any value from some finite set V.
- The MAP hypothesis tries to find the most probable target value as

$$
\begin{aligned}
v_{MAP} &= \underset{v_j \in V}{\operatorname{argmax}} P(v_j | a_1, a_2 \ldots a_n) \\
&= \underset{v_j \in V}{\operatorname{argmax}} \frac{P(a_1, a_2 \ldots a_n | v_j) P(v_j)}{P(a_1, a_2 \ldots a_n)} \\
&= \underset{v_j \in V}{\operatorname{argmax}} P(a_1, a_2 \ldots a_n | v_j) P(v_j)
\end{aligned}
$$

- $P(v_j)$ : frequency of target value $v_j$ in training examples
- $P(a_1, a_2, \ldots, an | vj)$ is difficult to calculate considering all possible combinations of $a_i$ leading to huge training set

# Naïve Bayes Learning

- **Assumption**: Features $a_1, a_2, \ldots, a_n$ are class conditionally independent
- So, for a given class $\boldsymbol{v_j}$, $\boldsymbol{P(a_1, a_2, \ldots, an|v_j) = P(a_1|v_j).P(a_2|v_j) \ldots P(a_n|v_j)}$

$$P(a_1, a_2 \ldots a_n|v_j) = \prod_i P(a_i|v_j)$$

**Naive Bayes classifier:**

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_i P(a_i|v_j)$$

- So, predict the class $\boldsymbol{v_j}$ for which $\boldsymbol{P(v_j).\prod_{i=1}^{n} P(a_i|v_j)}$ is maximized.

# Naïve Bayes Learning - Example

| Day | Outlook | Temperature | Humidity | Wind | PlayTennis |
|---|---|---|---|---|---|
| D1 | Sunny | Hot | High | Weak | No |
| D2 | Sunny | Hot | High | Strong | No |
| D3 | Overcast | Hot | High | Weak | Yes |
| D4 | Rain | Mild | High | Weak | Yes |
| D5 | Rain | Cool | Normal | Weak | Yes |
| D6 | Rain | Cool | Normal | Strong | No |
| D7 | Overcast | Cool | Normal | Strong | Yes |
| D8 | Sunny | Mild | High | Weak | No |
| D9 | Sunny | Cool | Normal | Weak | Yes |
| D10 | Rain | Mild | Normal | Weak | Yes |
| D11 | Sunny | Mild | Normal | Strong | Yes |
| D12 | Overcast | Mild | High | Strong | Yes |
| D13 | Overcast | Hot | Normal | Weak | Yes |
| D14 | Rain | Mild | High | Strong | No |

$$P(PlayTennis = yes) = 9/14 = .64$$

$$P(PlayTennis = no) = 5/14 = .36$$

| Outlook | Y | N |
|---|---|---|
| sunny | 2/9 | 3/5 |
| overcast | 4/9 | 0 |
| rain | 3/9 | 2/5 |

| Humidity | Y | N |
|---|---|---|
| high | 3/9 | 4/5 |
| normal | 6/9 | 1/5 |

| Tempreature | | |
|---|---|---|
| hot | 2/9 | 2/5 |
| mild | 4/9 | 2/5 |
| cool | 3/9 | 1/5 |

| Windy | | |
|---|---|---|
| Strong | 3/9 | 3/5 |
| Weak | 6/9 | 2/5 |

# Naïve Bayes Learning - Example

**New example:** $\langle Outlook = sunny, Temperature = cool, Humidity = high, Wind = strong \rangle$

$$v_{NB} = \underset{v_j \in \{yes,no\}}{\text{argmax}} \; P(v_j) \prod_i \; P(a_i|v_j)$$

$$= \underset{v_j \in \{yes,no\}}{\text{argmax}} \; P(v_j) \quad P(Outlook = sunny|v_j) P(Temperature = cool|v_j)$$

$$\cdot \; P(Humidity = high|v_j) P(Wind = strong|v_j)$$

$$v_{NB}(yes) = P(yes) \; P(sunny|yes) \; P(cool|yes) \; P(high|yes) \; P(strong|yes) = .0053$$

$$v_{NB}(no) = P(no) \; P(sunny|no) \; P(cool|no) \; P(high|no) \; P(strong|no) = .0206$$

$$v_{NB}(yes) = \frac{v_{NB}(yes)}{v_{NB}(yes)+v_{NB}(no)} = 0.205 \qquad v_{NB}(no) = \frac{v_{NB}(no)}{v_{NB}(yes)+v_{NB}(no)} = 0.795$$

# Bayes Optimal Classifier

"what is the **most probable hypothesis** given the training data?"

***vs.***

"what is the **most probable classification** of the new instance given the training data?"

**An intuitive example:**

Suppose the hypothesis space contains 3 hypothesis/models ($\boldsymbol{h_1}, \boldsymbol{h_2}$ *and* $\boldsymbol{h_3}$) that maps the input example $(\vec{x})$ to the target values

Objective is to find the model/hypothesis that best describes the mapping

Posteriori probabilities $P(h_1|\vec{x}) = 0.4, P(h_2|\vec{x}) = 0.3, P(h_3|\vec{x}) = 0.3$
$So, \boldsymbol{hMAP = h_1}$

- What if $h_1$ maps to $\vec{x}$ to **Yes** class and both $h_2$ *and* $h_3$ map $\vec{x}$ to **NO** class ??
- So, the probability of $\vec{x}$ being classified to Yes is 0.4 and NO being 0.6
- Most probable classification is different than the classification generated by $h_{MAP}$.

# Bayes Optimal Classifier

*Objective:*

$$\underset{v_j \in V}{\text{argmax}} \; \boldsymbol{P(vj|D)}$$

$$\Rightarrow \underset{v_j \in V}{\text{argmax}} \sum_{h_i = H} P(v_j|h_i).P(hi|D)$$

$$P(h_1|D) = .4, \;\; P(N|h_1) = 0, \;\; P(Y|h_1) = 1$$

$$P(h_2|D) = .3, \;\; P(N|h_2) = 1, \;\; P(Y|h_2) = 0$$

$$P(h_3|D) = .3, \;\; P(N|h_3) = 1, \;\; P(Y|h_3) = 0$$

$$\sum_{h_i \in H} P(Y|h_i)P(h_i|D) = .4$$

$$\sum_{h_i \in H} P(N|h_i)P(h_i|D) = .6$$

$$\underset{v_j \in \{Y, N\}}{\text{argmax}} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D) = N$$

# Estimating Probabilities

- Find $v_{NB}$ for a new example where *outlook=overcast*
  - $v_{NB}$(no) = 0 as $P(outlook = overcast|v_j = no) = 0$ (biased underestimate of the probability)
  - The probabilities calculated are called ***maximum-likelihood estimated of probabilities***

## Basian approach of *m-estimation of probabilities*

- For a given class $\boldsymbol{v_j}$, let's assume that $\boldsymbol{m_j}$ no. of examples of $\boldsymbol{v_j}$ class are added following the probability distribution $p$ on the attribute $a_i$.
- Ex: suppose $v_j$=*no*, and $a_i$ is *outlook* which can take on values {*overcast, sunny, rain*}.
- Consider $m_j$=*6* and ***p*** is uniform. So, 6 additional examples of ***no*** class are distributed over ***outlook*** as follows: 2 have value *outlook=overcast*, 2 have value *outlook=sunny*, 2 have value *outlook=rain*
- *Total # of 'no' class examples = 5 original + 6 augmented = 11*

$$P(outlook = overcast|v_j = no) = \frac{2}{11}$$

=> No probability can be 0

# Estimating Probabilities

## Basian approach of *m-estimation of probabilities*

$$P(a_i|v_j) = \frac{n_j^i + mjp}{n_j + mj}$$

- $n_j$ is the no. of original examples of $v_j$ class
- $m_j$ is the additional no. of examples to $v_j$ class
- $p = \frac{1}{k}$ if $a_i$ takes $k$ different values
- $n_j^i$ is the no of examples having attribute value $a_i$ out of $n_j$ examples of $v_j$ classs

# Estimating Probabilities

- **Basian approach of *m-estimation of probabilities***

$$P(v_j) = ?$$

$$P(v_j) = \frac{n_j + mj}{\sum_j n_j + mj}$$

***Estimate the class for the new example***

***<outlook = overcast, temperature = cool, humidity = high, wind = strong >***

# Minimum risk Naïve Bayes

- (A) "Misclassification of cancerous tumour as benign"
- (B) "Misclassification of benign tumour as cancerous"
- More risk in (A) than (B) !!
- (A) can cause more loss than (B) !!
- Let $\lambda(v_j)$ be the loss in misclassifying an example of class $v_j$ to some other class
- Given $\vec{X}$, the expected loss (*risk*) in classifying the example into class $v_k$ is

$$risk = \sum_{j \neq k} \lambda(v_j)\, P(v_j|\vec{X})$$

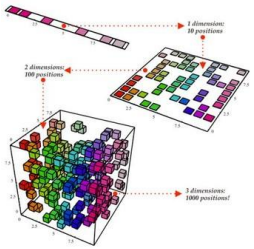
$$= \sum_{j} \lambda(v_j) P(v_j|\vec{X}) - \lambda(v_k) P(v_k|\vec{X})$$

- First term is constant and *risk* can be minimized by maximizing $\boldsymbol{\lambda(v_k) P(v_k|\vec{X})}$
- To minimize the risk in Naïve Bayes classifier, we maximize $\boldsymbol{\lambda(v_k).P(v_k)\prod_{i=1}^{n} P(a_i|v_k)}$

# Dimensionality Reduction

## Principal Component Analysis

Dr. Panthadeep Bhattacharjee
Dept. of CSE
NIT Rourkela



# Dimensionality Reduction

- Dimensionality reduction or dimension reduction is the process of reducing the number of random variables under consideration by obtaining a set of principal variables.
- It can be divided into feature selection and feature extraction.
  - Feature selection approaches try to find a subset of the original variables (also called features or attributes).
  - Feature projection or Feature extraction transforms the data in the high-dimensional space to a space of fewer dimensions.

# Large Dimensions

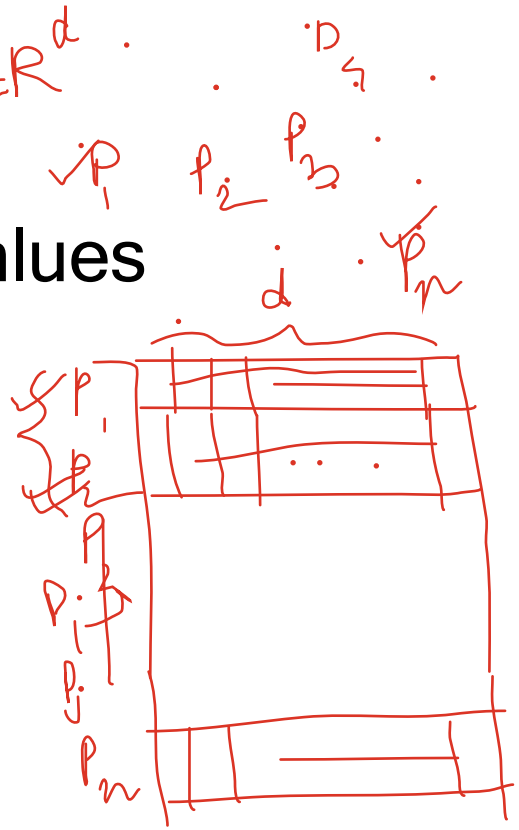
- Large number of features in the dataset is one of the factors that affect both the training time as well as accuracy of machine learning models. You have different options to deal with huge number of features in a dataset.
  - Try to train the models on original number of features, which take days or weeks if the number of features is too high.
  - Reduce the number of variables by merging correlated variables.
  - Extract the most important features from the dataset that are responsible for maximum variance in the output. Different statistical techniques are used for this purpose e.g. linear discriminant analysis, factor analysis, and principal component analysis.

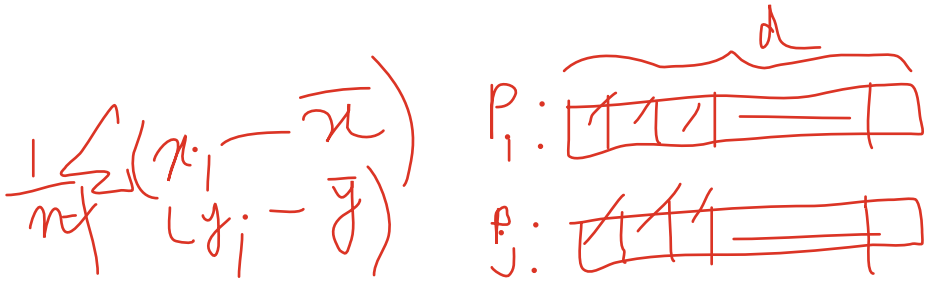
# Principal Component Analysis

- Principal component analysis, or PCA, is a statistical technique to convert high dimensional data to low dimensional data by selecting the most important features that capture maximum information about the dataset.
- The features are selected on the basis of variance that they cause in the output.
- The feature that causes highest variance is the first principal component. The feature that is responsible for second highest variance is considered the second principal component, and so on.
- It is important to mention that principal components do not have any correlation with each other.

# Steps in PCA

- Standardization
- Covariance Matrix Computation
- Computer Eigen vector and eigen values
- Feature vector
- Recast the Data Along the Principal Components Axes





# Standardization

- The aim of this step is to standardize the range of the continuous initial variables so that each one of them contributes equally to the analysis.
- More specifically, the reason why it is critical to perform standardization prior to PCA, is that the latter is quite sensitive regarding the variances of the initial variables.
- That is, if there are large differences between the ranges of initial variables, those variables with larger ranges will dominate over those with small ranges

# Standardization

- Mathematically, this can be done by subtracting the mean and dividing by the standard deviation for each value of each variable.

$$z = \frac{value - mean}{standard\ deviation}$$

- Once the standardization is done, all the variables will be transformed to the same scale.

# Covariance Matrix Computation

- The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other, or in other words, to see if there is any relationship between them.
- Because sometimes, variables are highly correlated in such a way that they contain redundant information.
- So, in order to identify these correlations, we compute the covariance matrix.

# Covariance Matrix Computation

$$\begin{bmatrix} Cov(x,x) & Cov(x,y) & Cov(x,z) \\ Cov(y,x) & Cov(y,y) & Cov(y,z) \\ Cov(z,x) & Cov(z,y) & Cov(z,z) \end{bmatrix}$$

# Covariance Matrix Computation

- What do the covariances that we have as entries of the matrix tell us about the correlations between the variables?
- It's actually the sign of the covariance that matters :
  - if positive then : the two variables increase or decrease together (correlated)
  - if negative then : One increases when the other decreases (Inversely correlated)
- Now, that we know that the covariance matrix is not more than a table that summaries the correlations between all the possible pairs of variables

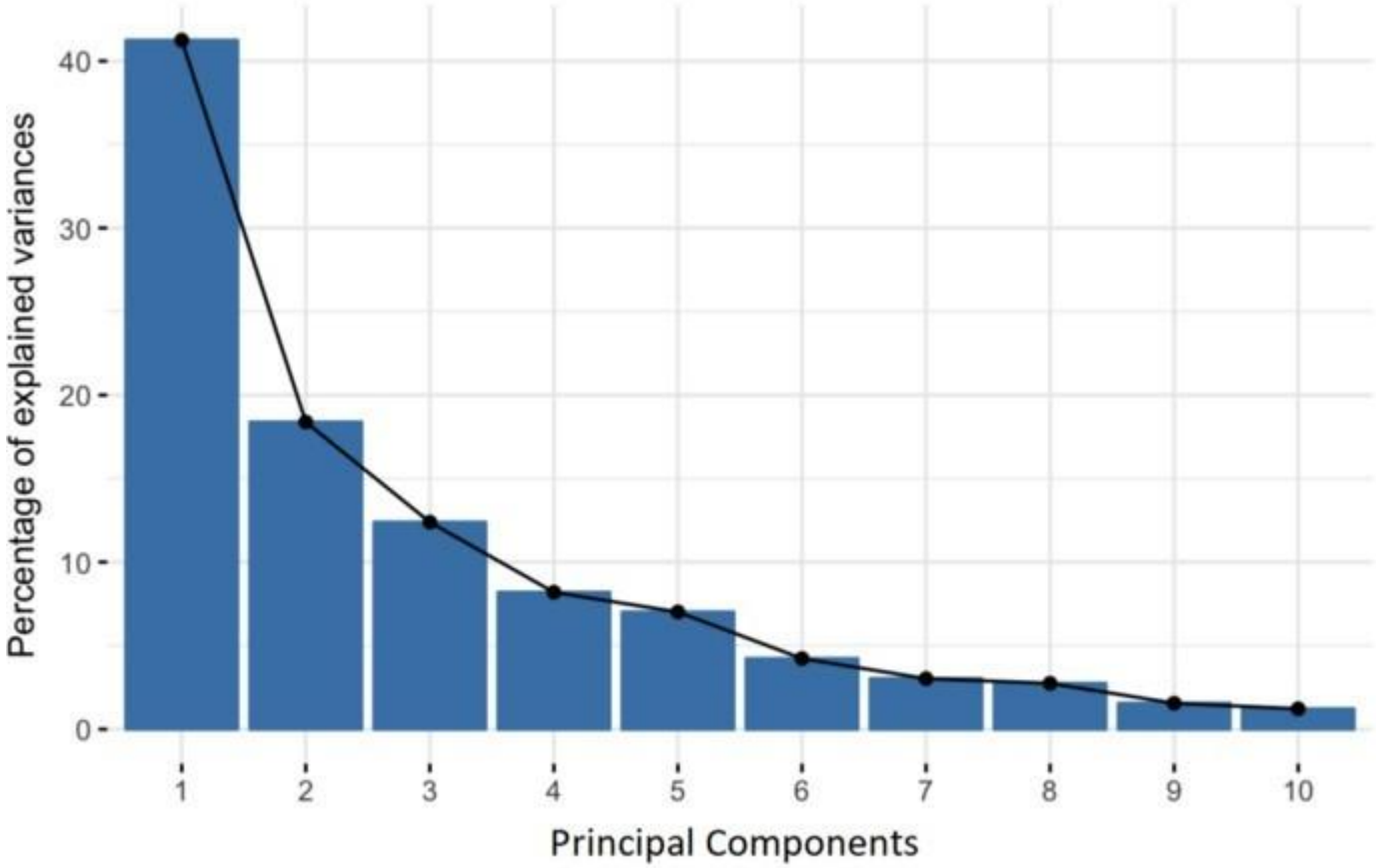
# Eigenvector and eigenvalues

- Eigenvectors and eigenvalues are the linear algebra concepts that we need to compute from the covariance matrix in order to determine the principal components of the data.
- Before getting to the explanation of these concepts, let's first understand what do we mean by principal components.

# Eigenvector and eigenvalues

- Principal components are new variables that are constructed as linear combinations or mixtures of the initial variables.
- These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components.
- So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on, until having something like shown in the scree plot below.

# Eigenvector and eigenvalues



# Principal Components

- Organizing information in principal components this way, will allow you to reduce dimensionality without losing much information, and this by discarding the components with low information and considering the remaining components as your new variables.
- An important thing to realize here is that, the principal components are less interpretable and don't have any real meaning since they are constructed as linear combinations of the initial variables.
- Geometrically speaking, principal components represent the directions of the data that explain a maximal amount of variance, that is to say, the lines that capture most information of the data.

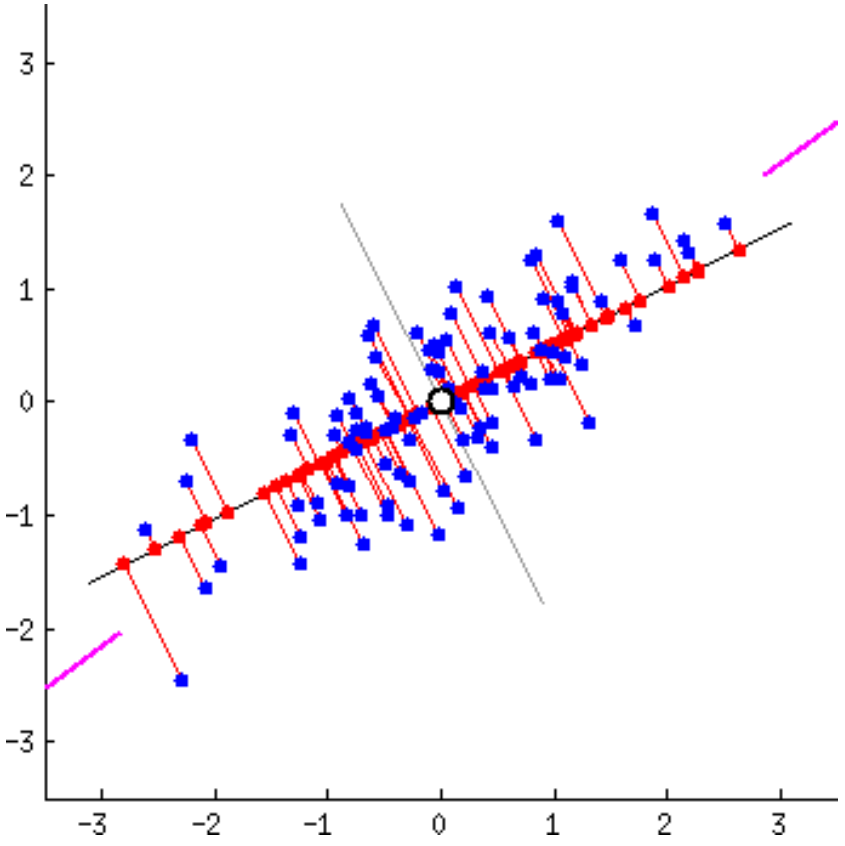
# Principal Components

- As there are as many principal components as there are variables in the data, principal components are constructed in such a manner that the first principal component accounts for the largest possible variance in the data set.

# Principal Components

- For example, let's assume that the scatter plot of our data set is as shown below, can we guess the first principal component ?
- Yes, it's approximately the line that matches the purple marks because it goes through the origin and it's the line in which the projection of the points (red dots) is the most spread out.
- Or mathematically speaking, it's the line that maximizes the variance (the average of the squared distances from the projected points (red dots) to the origin).

# Principal Components



# Example

- Let's suppose that our data set is 2-dimensional with 2 variables x,y and that the eigenvectors and eigenvalues of the covariance matrix are as follows:

$$v1 = \begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix} \qquad \lambda_1 = 1.284028$$

- $$v2 = \begin{bmatrix} -0.7351785 \\ 0.6778736 \end{bmatrix} \qquad \lambda_2 = 0.04908323$$

  If we rank the eigenvalues in descending order, we get λ1>λ2, which means that the eigenvector that corresponds to the first principal component (PC1) is v1 and the one that corresponds to the second component (PC2) isv2.

# Feature Vector

- As we saw in the previous step, computing the eigenvectors and ordering them by their eigenvalues in descending order, allow us to find the principal components in order of significance.
- In this step, what we do is, to choose whether to keep all these components or discard those of lesser significance (of low eigenvalues), and form with the remaining ones a matrix of vectors that we call Feature vector.
- So, the feature vector is simply a matrix that has as columns the eigenvectors of the components that we decide to keep.
- This makes it the first step towards dimensionality reduction, because if we choose to keep only p eigenvectors (components) out of n, the final data set will have only p dimensions.

# Example:

- Continuing with the example from the previous step, we can either form a feature vector with both of the eigenvectors v1 and v2:

$$\begin{bmatrix} 0.6778736 & -0.7351785 \\ 0.7351785 & 0.6778736 \end{bmatrix}$$

- Or discard the eigenvector v2, which is the one of lesser significance, and form a feature vector with v1 only:

$$\begin{bmatrix} 0.6778736 \\ 0.7351785 \end{bmatrix}$$

- Discarding the eigenvector v2 will reduce dimensionality by 1, and will consequently cause a loss of information in the final data set.

# Last step

- The aim is to use the feature vector formed using the eigenvectors of the covariance matrix, to reorient the data from the original axes to the ones represented by the principal components (hence the name Principal Components Analysis).
- This can be done by multiplying the transpose of the original data set by the transpose of the feature vector.

$$FinalDataSet = FeatureVector^T * StandardizedOriginalDataSet^T$$

# Advantages of PCA

- The training time of the algorithms reduces significantly with less number of features.
- It is not always possible to analyze data in high dimensions. For instance if there are 100 features in a dataset. Total number of scatter plots required to visualize the data would be 100(100-1)2 = 4950. Practically it is not possible to analyze data this way.

# Normalization of features

- It is imperative to mention that a feature set must be normalized before applying PCA. For instance if a feature set has data expressed in units of Kilograms, Light years, or Millions, the variance scale is huge in the training set. If PCA is applied on such a feature set, the resultant loadings for features with high variance will also be large. Hence, principal components will be biased towards features with high variance, leading to false results.
- Finally, the last point to remember before we start coding is that PCA is a statistical technique and can only be applied to numeric data. Therefore, categorical features are required to be converted into numerical features before PCA can be applied.

# Linear Discriminant Analysis

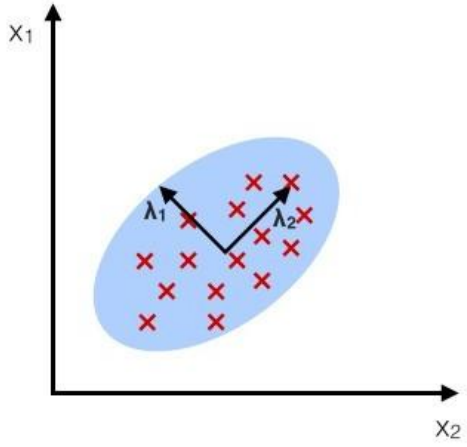
- Linear discriminant analysis (LDA), normal discriminant analysis (NDA), or discriminant function analysis is a generalization of Fisher's linear discriminant, a method used in statistics, pattern recognition, and machine learning to find a linear combination of features that characterizes or separates two or more classes of objects or events.
- The resulting combination may be used as a linear classifier, or, more commonly, for dimensionality reduction before later classification.

# How LDA is performed ?

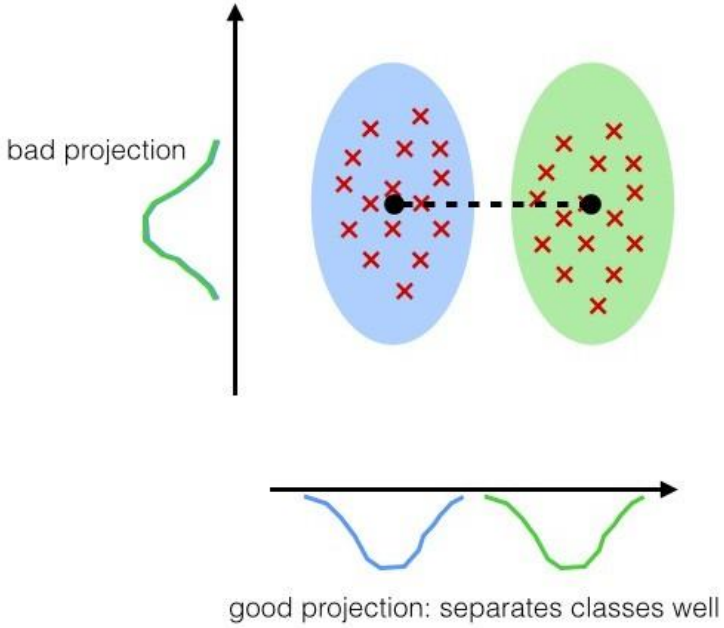
- Compute d-dimensional mean vectors for different classes from the dataset, where d is the dimension of feature space.
- Compute in-between class and with-in class scatter matrices.
- Compute eigen vectors and corresponding eigen values for the scatter matrices.
- Choose k eigen vectors corresponding to top k eigen values to form a transformation matrix of dimension d x k.
- Transform the d-dimensional feature space X to k-dimensional feature space X_lda via the transformation matrix.

# PCA vs. LDA

**PCA:**
component axes that maximize the variance




**LDA:**
maximizing the component axes for class-separation




Reference: sebastianraschka.com